# $\lambda_S^4$ — Parallel Composition:

## The $\parallel$ Constructor and the Lattice Theorem

Alexandre Zua Caldeira
Independent Researcher
azuacaldeira@gmail.com

March 2026

**Abstract**

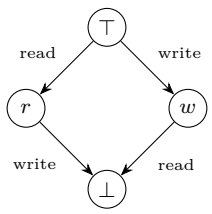
$\lambda_S^4$ completes the $\lambda_S$ hierarchy by adding the parallel constructor $S_1 \parallel S_2$ to session types. The parallel constructor models concurrent access to a shared session-typed object: two threads independently advance through sub-protocols $S_1$ and $S_2$, synchronising at completion. The key result is the *Product Construction*: $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$, ordered componentwise. This forces the state space to be a *lattice*—the central theorem of our research programme [1]. The term language gains **fork** (spawn concurrent branch) and **join** (synchronise) expressions. We prove preservation, progress, and the absence of data races for well-typed programs.

# 1 Introduction

Consider a file handle that supports concurrent reading and writing:

$$S = \&\{open : (\&\{read : \mathbf{end}\} \parallel \&\{write : \mathbf{end}\}) \,.\, \&\{close : \mathbf{end}\}\}$$

After *open*(), two threads may concurrently execute *read* and *write*. Both must complete before *close*() can be called. The state space of $\&\{read : \mathbf{end}\} \| \&\{write : \mathbf{end}\}$ is a $2 \times 2 = 4$-state product lattice:




This diamond is a lattice. The parallel constructor *always* produces a product lattice, which is the formal content of the Reticulate Theorem [1].

$\lambda_S^4$ adds:

1. **Parallel session types**: $S_1 \parallel S_2$.
2. **Product state spaces**: $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$.
3. **Fork expression**: **fork** $e_1 \parallel e_2$ spawns two concurrent sub-computations.
4. **Join / Wait**: after both branches complete, the continuation becomes available.
5. **Thread safety**: the type system prevents concurrent access to the same method unless the state space permits it.

**Structure.** Section 2 presents the syntax. Section 3 gives the type system. Section 4 presents the operational semantics. Section 5 proves soundness and thread safety. Section 6 works through examples. Section 7 completes the hierarchy.

# 2 $\lambda_S^4$ Syntax

## 2.1 Parallel Session Types

$$S \ ::= \ \&\{\ldots\} \ | \ \oplus\{\ldots\} \ | \ \mu X.\, S \ | \ X \ | \ S_1 \parallel S_2 \ | \ \mathbf{end}$$

Well-formedness (WF-Par) requires: each branch of $\parallel$ must terminate, no cross-branch recursion variables, and no nested $\parallel$ (see [2]).

## 2.2 Product State Spaces

**Definition 2.1** (Product Construction)**.** Given state spaces $\mathcal{L}(S_1)$ with states $Q_1$ and $\mathcal{L}(S_2)$ with states $Q_2$:

$$\mathcal{L}(S_1 \parallel S_2) = Q_1 \times Q_2$$

with componentwise transitions:

$$\begin{aligned} \delta((q_1, q_2), m) &= (q_1', q_2) && \text{if } \delta_1(q_1, m) = q_1' \\ \delta((q_1, q_2), m) &= (q_1, q_2') && \text{if } \delta_2(q_2, m) = q_2' \end{aligned}$$

Top: $(\top_1, \top_2)$. Bottom: $(\bot_1, \bot_2)$.

**Theorem 2.2** (Product Lattice [1])**.** *If $\mathcal{L}(S_1)$ and $\mathcal{L}(S_2)$ are lattices, then $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$ ordered componentwise is a lattice.*

This is why the $\parallel$ constructor *forces* lattice structure: the product of lattices is always a lattice.

## 2.3 Fork and Join

$$e \ ::= \ \ldots \ | \ \mathbf{fork}\ e_1 \parallel e_2 \ | \ \mathbf{join}\ x$$

**fork** $e_1 \parallel e_2$ splits execution: $e_1$ and $e_2$ run concurrently on the same session-typed object, each advancing through its branch of $S_1 \parallel S_2$. After both complete, the object is at $(\bot_1, \bot_2)$.

## 2.4 Continuation Types

After a parallel block completes, the protocol continues: $(S_1 \parallel S_2)\,.\,S_3$. The object transitions from the product state space to $\mathcal{L}(S_3)$.

# 3 Type System

All rules from $\lambda_S^3$ carry over. Two new rules.

$$\textsc{T-Fork}\ (\star)\ \frac{\begin{array}{c}\Gamma(x) = C \mathbin{@} (s_1, s_2) \qquad \mathcal{CT};\ \Gamma[x \mapsto C \mathbin{@} (s_1, \top_2)] \vdash e_1 : T_1 \dashv \Gamma_1 \\ \mathcal{CT};\ \Gamma[x \mapsto C \mathbin{@} (\top_1, s_2)] \vdash e_2 : T_2 \dashv \Gamma_2 \qquad \Gamma_1(x) = C \mathbin{@} (\bot_1, \top_2) \qquad \Gamma_2(x) = C \mathbin{@} (\top_1, \bot_2)\end{array}}{\mathcal{CT};\ \Gamma \vdash \mathbf{fork}\ e_1 \parallel e_2 : \mathbf{Unit} \dashv \Gamma[x \mapsto C \mathbin{@} (\bot_1, \bot_2)]}$$

T-Fork: each branch is checked independently against its component of the product state. Branch $e_1$ sees only the $S_1$ transitions; $e_2$ sees only $S_2$. After both complete, $x$ is at $(\bot_1, \bot_2)$—the bottom of the product.

**Remark 3.1** (Thread safety)**.** The componentwise typing ensures that $e_1$ and $e_2$ access *disjoint* methods: $e_1$ calls methods from $S_1$, $e_2$ calls methods from $S_2$. No data race is possible because the methods are structurally separated by the parallel constructor.

$$\textsc{T-Join}\ (\star)\ \frac{\Gamma(x) = C \mathbin{@} (\bot_1, \bot_2) \qquad (\bot_1, \bot_2) = \bot_{\|}}{\mathcal{CT};\ \Gamma \vdash \textbf{join}\ x : \textbf{Unit} \dashv \Gamma}$$

T-Join: verifies that both branches have completed. After join, the protocol may continue via a continuation type.

# 4 Operational Semantics

$$\textsc{R-Fork}\ (\star)\ \frac{}{\langle H,\ \sigma,\ \textbf{fork}\ e_1 \parallel e_2\rangle \longrightarrow \langle H,\ \sigma,\ e_1 \parallel e_2\rangle} \qquad \textsc{R-Par-Left}\ (\star)\ \frac{\langle H,\ \sigma,\ e_1\rangle \longrightarrow \langle H',\ \sigma',\ e_1'\rangle}{\langle H,\ \sigma,\ e_1 \parallel e_2\rangle \longrightarrow \langle H',\ \sigma',\ e_1' \parallel e_2\rangle}$$

$$\textsc{R-Par-Right}\ (\star)\ \frac{\langle H,\ \sigma,\ e_2\rangle \longrightarrow \langle H',\ \sigma',\ e_2'\rangle}{\langle H,\ \sigma,\ e_1 \parallel e_2\rangle \longrightarrow \langle H',\ \sigma',\ e_1 \parallel e_2'\rangle} \qquad \textsc{R-Par-Done}\ (\star)\ \frac{}{\langle H,\ \sigma,\ v_1 \parallel v_2\rangle \longrightarrow \langle H,\ \sigma,\ \textbf{skip}\rangle}$$

R-Fork unfolds into a parallel composition. R-Par-Left/Right reduce either branch (nondeterministic interleaving). R-Par-Done: when both branches are values, the parallel composition completes.

**Remark 4.1.** The state store $\sigma$ maps locations to product states $(s_1, s_2)$. A left-branch method call updates $s_1$; a right-branch call updates $s_2$. The componentwise update is exactly the product transition function.

# 5 Soundness and Thread Safety

**Theorem 5.1** (Preservation)**.** *Well-typed configurations reduce to well-typed configurations.*

*Proof.* New cases:

- R-Fork: by T-Fork premises.
- R-Par-Left/Right: by IH on the sub-reduction. The other branch's component is unchanged.
- R-Par-Done: result is **skip** : **Unit**. □

**Theorem 5.2** (Progress)**.** *Well-typed non-values can reduce.*

**Theorem 5.3** (Thread Safety)**.** *In a well-typed $\lambda_S^4$ program, concurrent branches of a* **fork** *expression access disjoint sets of methods.*

*Proof.* By T-Fork: $e_1$ is typed against $S_1$ transitions; $e_2$ against $S_2$ transitions. Since $S_1 \parallel S_2$ requires disjoint method sets (by WF-Par), no method is called by both branches. □

**Corollary 5.4** (No Data Races)**.** *Well-typed $\lambda_S^4$ programs have no data races on session-typed objects.*

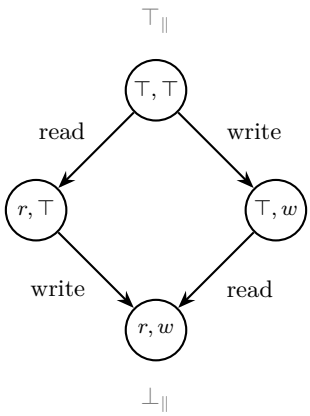



Figure 1: Product state space for $\&\{\mathit{read} : \mathbf{end}\} \parallel \&\{\mathit{write} : \mathbf{end}\}$. The diamond is a lattice.

Table 1: The complete $\lambda_S$ hierarchy.

| **Calc.** | **Session types** | **Terms** | **Terminates?** | **Key property** |
|---|---|---|---|---|
| $\lambda_S^0$ | &, end | let, call | Yes | Straight-line typestate |
| $\lambda_S^1$ | &, end | +const | Yes | Orthogonality of returns |
| $\lambda_S^2$ | &, $\oplus$, end | +match | Yes | Enum returns $\to$ branches |
| $\lambda_S^3$ | $+\mu$ | +while | No | Cyclic state spaces, SCCs |
| $\lambda_S^4$ | $+\parallel$ | +fork/join | No | **Product lattice** |

# 6 Worked Examples

## 6.1 Concurrent Read/Write

$$S = \&\{\mathit{open} : (\&\{\mathit{read} : \mathbf{end}\} \parallel \&\{\mathit{write} : \mathbf{end}\}) \,.\, \&\{\mathit{close} : \mathbf{end}\}\}$$

Client:

$$\begin{aligned} e = \; & \mathbf{let}\ f = \mathbf{new}\ \mathrm{FH}()\ \mathbf{in} \\ & \mathbf{let}\ _ = f.\mathit{open}()\ \mathbf{in} \\ & \mathbf{fork}\ f.\mathit{read}() \parallel f.\mathit{write}() \\ & f.\mathit{close}() \end{aligned}$$

The product state space after *open*: $\{(\top, \top), (r, \top), (\top, w), (r, w)\}$ where $r = \bot_1$, $w = \bot_2$. This $2 \times 2$ diamond is a lattice.

# 7 The Complete $\lambda_S$ Hierarchy

$\lambda_S^4$ completes the hierarchy (Table 1). Together, the five calculi provide a formal foundation for session types on objects with:

- Straight-line code ($\lambda_S^0$)
- Scalar returns ($\lambda_S^1$)
- Selection/branching ($\lambda_S^2$)
- Loops/recursion ($\lambda_S^3$)
- Concurrency ($\lambda_S^4$)

The product lattice construction in $\lambda_S^4$ is the formal content of the Reticulate Theorem: the $\|$ constructor makes lattice structure *necessary* (not merely nice), because the product of lattices is always a lattice. This connects the algebraic structure of session type state spaces to the practical requirement of thread-safe concurrent access.

## References


[1] Alexandre Zua Caldeira. Session types as algebraic reticulates: The lattice theorem (step 5), 2026. Submitted to ICE 2026.

[2] Alexandre Zua Caldeira. State-space construction for binary session types (step 1), 2026. arXiv preprint.